

ISDASAFE — ENGINEERING REFERENCE

How the app is put together

A map of every folder in the codebase — what lives inside it, and why it exists — written for anyone who needs to understand the shape of the system without reading the code itself.

SUBJECT	FRAMEWORK	SCOPE
IsdaSafe pond-monitoring app	Flutter (one codebase, six platforms)	Full repository structure

MAP

The structure, at a glance

Everything below traces back to one of these branches. Solid folders hold real app logic; the rest is scaffolding, tests, or reference material.

```
isdasafe/  
  android/ · ios/ · windows/ · macos/ · linux/ · web/ platform shells  
  lib/ the app itself  
    screens/ (incl. auth/)  
    widgets/  
    models/  
    providers/  
    services/  
    theme/  
  test/ automated tests  
    support/  
  supabase/functions/ backend functions  
  docs/ setup guides  
  pubspec.yaml, vercel.json, README.md ... root config
```

01 Platform shells

Flutter lets one codebase run on every device — these six folders are the packaging each platform needs to install and launch it. No app logic lives here.

> `android/ ios/ windows/ macos/ linux/ web/`

auto-generated

02 lib/ — the app itself

Every screen, every rule, every piece of shared visual language lives in one of these six folders. Each has exactly one job.

▼ `screens/` Every full page a user sees

Dashboard, Map, Analytics, Settings, Notifications — plus an `auth/` subfolder holding Login, Register, and phone verification.

- <code>dashboard_screen.dart</code>	- <code>pond_map_screen.dart</code>	- <code>analytics_screen.dart</code>
- <code>settings_screen.dart</code>	- <code>auth/login_screen.dart</code>	- <code>auth/register_screen.dart</code>

> `widgets/` Reusable pieces of a screen

> `models/` What the app's data looks like

> `providers/` The app's live, shared state

> `services/` Talking to the outside world, and the app's rules

> `theme/` Shared look and feel

03 test/ — the safety net

Mirrors `lib/` folder-for-folder, plus one addition: `support/`, shared helpers used only by the tests themselves.

> test/

never ships to users

04 supabase/ — backend functions

Code that runs on the server, not on a user's phone.

> supabase/functions/

2 functions

05 docs/ & root files

Reference material and project-wide configuration — not part of the running app, but part of running the project.

> docs/

2 guides

> pubspec.yaml · vercel.json · README.md

NOTE

build/ is not part of the source — it's temporary, regenerated output produced every time the app is compiled. It's never something to review, commit permanently, or hand to a client.